

Cost-Efficient Domain RAG with Fine-Tuned Matryoshka Embeddings

1. Executive Summary

The architectural paradigm of generative artificial intelligence has decisively shifted toward Retrieval-Augmented Generation (RAG) to ground Large Language Models (LLMs) in factual, domain-specific contexts. However, as enterprise RAG systems scale to encompass millions or billions of documents, the infrastructural burden associated with vector storage and the computational latency of similarity search have emerged as critical deployment bottlenecks. High-dimensional embeddings, while possessing rich semantic density, impose substantial memory footprints and exacerbate the cost of cosine similarity calculations.

This research and implementation blueprint delineates a rigorous methodology to resolve this vector bloat through the application of Matryoshka Representation Learning (MRL). MRL fundamentally alters the training dynamics of embedding models, compelling the neural network to encode the most critical semantic variance into the earliest dimensions of the output vector. Consequently, the architecture permits the truncation of embeddings at inference time—for instance, compressing a 768-dimensional vector to 128 dimensions—without a catastrophic degradation in downstream retrieval quality¹.

Because truncating generic, pre-trained models often yields unacceptable accuracy loss for highly specialized vocabularies, this project mandates the domain-specific fine-tuning of a base model. The proposed system will leverage contrastive learning via MultipleNegativesRankingLoss paired with a nested MatryoshkaLoss objective. The final experimental outcomes will explicitly quantify the precise trade-offs between embedding dimensionality, storage requirements, retrieval latency, and ranking accuracy as measured by Recall and Normalized Discounted Cumulative Gain (nDCG).

For an applied AI Engineer, demonstrating empirical mastery over these optimization techniques is highly consequential. The contemporary tech market heavily premiums engineering talent capable of bridging the gap between theoretical model training and cost-efficient production deployment. Mastery of PyTorch fine-tuning, advanced loss functions, and hardware-aware architectural design signals a readiness for senior roles, where engineers are expected to construct systems that balance algorithmic accuracy with infrastructural economy³.

2. Simple Explanation

To conceptualize the mechanics of this system, one must understand how unstructured text is mathematically transformed and queried. Embeddings operate as dense arrays of continuous numerical values—vectors—that represent the semantic meaning of text. The length of this numerical array is referred to as its vector dimension; a 768-dimensional vector contains 768 distinct floating-point numbers. The foundational mechanism for searching these vectors is cosine similarity, a mathematical operation that calculates the angle between two vectors in high-dimensional space. Smaller angles indicate closer semantic alignment, allowing the

system to perform dense retrieval—searching for information by conceptual meaning rather than relying on exact keyword matching.

In the context of RAG, dense retrieval is tasked with fetching the most relevant documents from a massive database to provide context for an LLM to formulate an accurate answer. However, generic models often struggle with specialized jargon. Fine-tuning solves this by continuing the training process of a pre-trained model on specialized data, thereby adapting it to a specific domain. This training relies on contrastive learning, a pedagogical approach for neural networks where the model is taught by pulling positive examples (relevant query-document pairs) closer together in the vector space while pushing negative examples (irrelevant documents) further apart.

To optimize this training, modern systems utilize MultipleNegativesRankingLoss, an efficient technique that repurposes the positive documents of other queries within the same training batch as negative examples for the current query, accelerating the learning process⁶.

Matryoshka Representation Learning subsequently steps in to reorganize how the model stores this learned information. Named after Russian nesting dolls, MRL forces the model to pack its core semantic understanding into the first few numbers of the vector. This allows the system to lop off the tail end of the vector, drastically reducing the storage space required on disk and in RAM, while simultaneously accelerating the mathematical comparisons needed for retrieval. It is vital to note, however, that MRL does not reduce the initial computational cost of generating the embedding; the neural network must still process the text using its full architecture before the resulting vector is truncated¹.

The holistic architecture flows seamlessly from raw data to final inference. Documents are processed by the embedding model, which is fine-tuned using contrastive learning and MultipleNegativesRankingLoss. The addition of Matryoshka training ensures the vectors support multiple embedding dimensions. These truncated vectors are stored in a specialized vector database like FAISS. During inference, a user query undergoes the same truncation, triggering a high-speed retrieval process. The results are evaluated for accuracy, and ultimately fed into the RAG pipeline to produce an LLM-generated answer.

3. Problem Statement

Large-scale RAG deployments routinely index millions of text chunks. The default dimensionality for modern open-source embedding models ranges from 768 to 1024, while proprietary enterprise models often exceed 1536 or 3072 dimensions. This high dimensionality imposes a severe tax on infrastructure.

The precise technical problem investigated by this project is whether a domain-adapted, fine-tuned Matryoshka embedding model can yield significantly smaller representations (e.g., 128 or 256 dimensions) while retaining a dominant percentage of the retrieval quality exhibited by a full 768-dimensional baseline.

To properly scope the problem, the operational costs of a RAG system must be strictly categorized. Embedding generation cost refers to the forward-pass compute required by the Transformer model to encode text; MRL provides no savings here, as the full network must execute. Vector storage cost dictates the memory (RAM/VRAM) and persistent disk required by the vector index; MRL directly and linearly reduces this cost. Vector search and processing

cost involves the CPU or GPU cycles spent computing cosine similarities across the index; MRL reduces this by requiring fewer floating-point operations per comparison. Retrieval latency is the wall-clock time required to return candidates, which improves under MRL due to reduced memory bandwidth pressure⁷. Reranking cost, the secondary phase where a computationally heavy cross-encoder scores the retrieved candidates, remains unaffected by MRL⁸. Finally, LLM generation cost is strictly a function of the generative model and remains completely isolated from the embedding dimensionality.

4. Project Objectives

The execution of this project is anchored by nine empirically measurable objectives designed to validate the central thesis.

The primary objective is the construction of a reproducible baseline RAG retrieval system utilizing a pre-trained, full-dimensional open-source embedding model to establish a performance floor. Following this, the second objective entails the programmatic fine-tuning of the base embedding model specifically tuned for domain-specific retrieval using a curated dataset. The third objective is the successful integration of contrastive learning via the MultipleNegativesRankingLoss function to create a mathematically separated semantic space. The core research thesis is addressed in the fourth objective, which requires the wrapping of the contrastive loss function within a MatryoshkaLoss objective to enforce multi-granularity representation learning across nested vector subsets. With the model trained, the fifth objective is the extraction and structural evaluation of these embeddings across multiple nested dimensions, specifically 768, 512, 256, 128, and 64. The sixth and seventh objectives are analytical: measuring retrieval quality using standard Recall@K and nDCG metrics, followed by profiling empirical vector storage requirements and computational retrieval latency. The ultimate goal of the research, formulated as the eighth objective, is to determine the optimal quality-to-cost trade-off point for the targeted domain, identifying the exact dimension where storage savings are maximized before retrieval degradation becomes unacceptable. The final objective is the integration of this optimal, truncated embedding model into a functioning, end-to-end Python RAG pipeline.

5. Required Concepts

A deep sequential understanding of the underlying theory is paramount for the architectural decisions required in this project. The following conceptual sequence builds the necessary intuition from basic linear algebra to advanced deep learning loss optimization.

The foundation begins with Embeddings, which serve as dense continuous vector representations generated by Transformer networks, fundamentally translating human textual semantics into machine-searchable mathematics. These Vector representations plot concepts as coordinates in a high-dimensional space where Euclidean distance or angle equates to semantic similarity, forming the core logic of the retrieval engine. Cosine similarity calculates the angle between these vectors via their dot product normalized by magnitude, acting as the primary mathematical search metric. This leads into Dense retrieval, the paradigm of searching across this dense mathematical matrix rather than relying on sparse inverted indices like BM25, effectively overcoming the vocabulary mismatch inherent in traditional search. RAG retrieval

represents the practical application of this search, acting as the first-stage fetcher of context in generative pipelines.

Transitioning to implementation, PyTorch fundamentals must be grasped as the dynamic computational graph framework required for manipulating model weights. Neural network training explains the iterative adjustment of these weights via backpropagation and gradient descent, which is the exact mechanism driving Fine-tuning—the process of adapting generalized pre-trained weights to highly specialized domain distributions. Loss functions serve as the mathematical penalty functions defining exactly what the neural network learns to optimize.

The specific training paradigm utilized is Contrastive learning, which learns representations by contrasting positive semantic pairs against negative ones, generating a well-separated embedding space. This requires Positive/negative pairs, specifically query-document pairings that serve as the ground truth. To optimize this, In-batch negatives are employed, an elegant technique treating all other positive documents in a training batch as negatives for a given query, which massively increases computational training efficiency. This is mathematically formalized in MultipleNegativesRankingLoss, a loss function that applies cross-entropy to the similarity scores of these in-batch negatives, serving as the standard for training modern dense retrievers⁶.

The defining innovation of the project is Matryoshka Representation Learning, which forces the network to solve the contrastive loss function at multiple explicit vector truncation points simultaneously. This is implemented via MatryoshkaLoss, a wrapper class that applies the base MNRL to nested sub-dimensions¹.

To validate the model, Retrieval evaluation employs statistical metrics to measure system efficacy. Recall@K measures the fraction of relevant documents successfully retrieved in the top K results, acting as the most critical gating metric for RAG. MRR (Mean Reciprocal Rank) averages the reciprocal of the rank of the first relevant document, penalizing models for pushing correct answers lower down the list. nDCG (Normalized Discounted Cumulative Gain) provides graded relevance scoring that heavily weights the absolute top positions. Finally, Latency/storage evaluation profiles RAM, disk usage, and inference times in milliseconds, ultimately proving the cost-efficiency objective of the MRL implementation.

6. Mathematical Foundations

The mathematics governing this retrieval system dictate exactly how semantic similarity is

quantified and subsequently optimized by the neural network. A vector V residing in an n -dimensional space is mathematically represented as a one-dimensional array

$V = [v_1, v_2, \dots, v_n]$. For standard transformer-based embeddings, n typically equals 768.

The fundamental operation between two such vectors is the dot product, calculated as

$$\sum_{i=1}^n A_i B_i$$

Cosine similarity normalizes the dot product by the magnitude of the vectors to isolate the angular difference, removing the influence of vector length. It is defined as:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

In modern embedding models, vectors are frequently L2-normalized so that their magnitude is exactly 1. When vectors are L2-normalized, their cosine similarity simplifies entirely to their dot product, vastly accelerating hardware-level matrix multiplications⁷.

During training, the Contrastive Learning Objective must update the model's weights to maximize this similarity for relevant pairs and minimize it for irrelevant pairs. For a given anchor

query q_i , a positive relevant document d_i^+ , and a set of in-batch negative documents $d_{i,j}^-$,

the model calculates a matrix of similarity scores S . The Multiple Negatives Ranking Loss (MNRL) applies the softmax function to these similarity scores, treating the batch as a multi-class classification problem where the model must identify the single positive document out of the batch. The loss is computed via cross-entropy:

$$\mathcal{L}_{MNRL} = -\log \frac{\exp(S(q_i, d_i^+))}{\exp(S(q_i, d_i^+)) + \sum_j \exp(S(q_i, d_{i,j}^-))}$$

The Matryoshka Training Objective modifies this base loss. Instead of calculating MNRL solely

on the full d -dimensional vector, MRL calculates the loss over a predefined set of truncated

dimensions $M = \{d_1, d_2, \dots, d_m\}$ (for example, $\{64, 128, 256, 768\}$), where

$d_1 < d_2 < d_m$. The total loss propagated backward through the network is a weighted sum of the individual losses calculated at each sub-dimension:

$$\mathcal{L}_{Matryoshka} = \sum_{m \in M} c_m \mathcal{L}_{MNRL}^{(m)}$$

The constants c_m represent importance weights. While they can be scaled to heavily protect the full dimension, setting all weights to 1.0 has been empirically shown to effectively force the network to push its most critical semantic variance into the earliest dimensions².

7. Dataset Research

Selecting the appropriate dataset is the most critical logistical decision for a Google Colab

environment constrained by single-GPU memory limits and a strict 7-day development window. The dataset must possess a high-quality domain shift to prove the efficacy of fine-tuning, yet remain computationally tractable.

Dataset Name	Domain	Corpus Size	Training Split	Qrels Available	Colab Feasibility
MS MARCO	General Web / QA	8.8 Million	500k+ pairs	Yes	Low (Corpus indexing alone exceeds typical Colab RAM limits; requires massive subsampling).
Natural Questions	Wikipedia QA	2.6 Million	100k+ pairs	Yes	Low (Extremely large corpus, generic domain).
SciFact	Scientific / Medical	5.1 Thousand	800+ queries	Yes	High (Easily fits in RAM, highly specialized vocabulary proves domain adaptation).
FiQA	Financial QA	57 Thousand	5k+ queries	Yes	Medium-High (Manageable corpus size, distinct

					financial domain shift).
NFCorpus	Medical	3.6 Thousand	3k+ queries	Yes	High (Very small, excellent for rapid iteration).
HotpotQA	Multi-hop Web	5.2 Million	90k+ pairs	Yes	Low (Too large, and multi-hop reasoning is out of scope for standard first-stage RAG).

Datasets curated under the BEIR (Benchmarking IR) umbrella format are highly preferred because they provide standardized Hugging Face data structures, separating the corpus, queries, and relevance judgments (qrels) clearly². Attempting to use MS MARCO for a 7-day Colab project frequently results in CUDA Out-Of-Memory (OOM) errors during the FAISS indexing phase unless aggressive, statistically questionable subsetting is employed.

8. Test Data Strategy

To ensure scientific validity without overwhelming Colab's ephemeral resources, the project relies on a strict separation of training and evaluation data, governed by predefined relevance judgments (qrels).

The training data must consist exclusively of pairs of (query, positive_document). Because MNRL utilizes in-batch negatives, explicitly mined hard negatives are not strictly required for the MVP pipeline, though adding them can marginally improve performance². A validation data holdout set is required to monitor the loss curve and prevent overfitting—a common failure mode when fine-tuning on small domain datasets. The test data must strictly not overlap with the training data to prevent evaluation leakage.

If a massive dataset were utilized, creating a statistically sensible subset would require random uniform sampling of the queries, followed by a filtering mechanism to ensure that the corpus only includes the positive documents for those sampled queries alongside a large random selection of distractors. However, utilizing a naturally small, self-contained dataset like SciFact

guarantees that the full, unadulterated corpus can be indexed and searched rapidly during the evaluation phase, preserving the integrity of the Recall@K metrics.

9. Dataset Recommendation

Based on the 7-day Colab constraint and the imperative to prove domain adaptation empirically, the primary and secondary datasets are formally recommended.

PRIMARY DATASET: SciFact (BEIR format)

- **Training/Validation/Testing:** The dataset provides distinct splits via the Hugging Face BeIR/scifact repository.
- **Justification:** The corpus is exceptionally compact (~5,183 documents). Building the vector index takes seconds rather than hours, allowing for rapid debugging of the PyTorch loop. More importantly, the vocabulary is heavily skewed toward scientific and medical claims. Fine-tuning a general-purpose model on this data will yield immediate, highly visible improvements in retrieval metrics, conclusively proving the efficacy of the pipeline.

OPTIONAL SECOND DATASET: FiQA (Financial Question Answering)

- **Purpose:** Out-of-domain generalization evaluation.
- **Justification:** Evaluating the SciFact-trained model on FiQA can test whether fine-tuning heavily on scientific literature caused "catastrophic forgetting" of general or financial concepts, establishing the boundary conditions and limitations of the resulting embedding model.

10. Google Colab Environment

Google Colab provides a volatile but powerful environment that dictates the architectural engineering of the training pipeline. The recommended Python version is 3.10+, utilizing PyTorch 2.0+ to take advantage of advanced compilation features.

For standard free-tier execution, the T4 GPU provides 16GB of VRAM. This is sufficient for fine-tuning base models (e.g., 100M-300M parameters) with a reasonable batch size (16 to 64). If Colab Pro is available, utilizing an L4 or A100 GPU allows for significantly larger batch sizes, which directly improves MNRL performance by increasing the pool of in-batch negatives². System RAM requirements are minimal for SciFact, easily fitting within Colab's standard 12GB allocation.

Because Colab environments reset upon idle timeouts, persistent storage is non-negotiable. Google Drive must be mounted at the start of every notebook. All downloaded datasets, intermediate model checkpoints, and final evaluation dataframes must be written directly to a structured directory (e.g., /content/drive/MyDrive/Matryoshka-RAG/).

To avoid CUDA GPU Out-Of-Memory (OOM) errors, the pipeline must implement rigorous memory management. Explicitly calling `torch.cuda.empty_cache()` between major operations is advised. Mixed precision training (`torch.cuda.amp.autocast`) should be enabled, utilizing FP16 or BF16 datatypes to halve memory consumption and increase throughput¹². If OOM persists, the batch size must be reduced, compensating via gradient accumulation—performing multiple forward passes and accumulating gradients before stepping the optimizer¹². GPU memory can be manually verified using the `nvidia-smi` command.

11. Model Selection

The chosen embedding model must natively support the sentence-transformers library, standard PyTorch fine-tuning loops, and Matryoshka dimensionality wrapper classes.

PRIMARY MODEL: BAAI/bge-base-en-v1.5

- **Architecture:** 109M parameters, 768 native dimensions.
- **Justification:** It currently stands as one of the strongest open-source baseline models for English retrieval¹. Because it is already a highly capable general retriever, improving its metrics on a specific domain (SciFact) using MRL will serve as a rigorous, undeniable test of the fine-tuning methodology. It fits comfortably inside a T4 GPU's 16GB VRAM during training.

BACKUP MODEL: sentence-transformers/all-MiniLM-L6-v2

- **Architecture:** 22M parameters, 384 native dimensions.
- **Justification:** Extremely lightweight and fast. This is the ideal fallback if the Colab environment faces severe memory strictures. However, because its maximum dimension is only 384, it allows for fewer meaningful MRL truncation steps (e.g., restricted to testing 256, 128, and 64) compared to a 768-dimensional model.

12. Baseline System

Before introducing any fine-tuning complexities or MRL wrappers, a rigid baseline system must be established to quantify subsequent improvements. The baseline flow operates as follows: The Pretrained bge-base model is loaded into memory. It processes the entire SciFact document corpus, generating a matrix of 768-dimensional document embeddings. These embeddings are pushed into a vector database (FAISS) utilizing a FlatL2 or InnerProduct index. Subsequently, the model processes the test set queries into query embeddings. The system performs a highly optimized similarity search via FAISS to retrieve the Top-K documents for each query. Finally, the evaluation module compares the retrieved candidate lists against the gold-standard qrels to calculate baseline scores.

This exact process is most efficiently implemented utilizing the InformationRetrievalEvaluator native to the sentence-transformers library, which automatically manages the indexing and metric computation¹⁰. The scores generated here represent the absolute performance floor for the project.

13. Fine-Tuning

Fine-tuning adapts the pre-trained weights of the transformer to the syntactical and semantic nuances of the scientific domain. The training data must be formatted into a PyTorch DataLoader yielding instances of InputExample(texts=[query, positive_document]). Hyperparameter selection in Colab is tightly constrained. The batch size should be maximized up to the VRAM limits—typically 32 or 64 for a base model on a T4 GPU. Larger batch sizes exponentially increase the number of in-batch negatives, drastically improving the contrastive learning gradient signal².

The optimizer of choice must be AdamW. Unlike standard Adam, AdamW decouples weight decay from the gradient calculation, applying the regularization penalty directly to the weights.

This distinction is critical for stabilizing the training of complex Transformer architectures¹². The learning rate should be set conservatively, around 2×10^{-5} , to prevent catastrophic forgetting of the pre-trained knowledge. A learning rate scheduler is imperative; CosineAnnealingLR combined with a linear warmup phase (spanning the first 10% of total training steps) ensures the model does not take chaotic gradient steps early in training before settling into a smooth decay curve¹³. Given the small size of the SciFact dataset, training should not exceed 1 to 3 epochs to avoid severe overfitting.

14. Contrastive Learning

Contrastive learning operates on the principle of relative spatial arrangement within the embedding matrix. Consider the query: *"How does hypertension influence ischemic events?"* The positive document might state: *"Chronic high blood pressure is shown to significantly elevate the probability of ischemic stroke."* An in-batch negative document might cover a completely unrelated topic: *"Quantum entanglement describes dependent particle states."* During the forward pass, the model projects all three texts into the 768-dimensional space. The contrastive loss function calculates the cosine similarity between the vectors. The optimizer then calculates gradients that mathematically adjust the neural network's self-attention weights. These adjustments force the query vector and the positive document vector closer together (driving the angle toward 0 degrees), while simultaneously pulling the query vector and the negative document vector apart. Over thousands of steps, this transforms the relatively isotropic pre-trained embedding space into a highly clustered, domain-specific semantic topography.

15. MultipleNegativesRankingLoss

MultipleNegativesRankingLoss (MNRL) is a highly optimized variant of contrastive loss designed for large-scale information retrieval. Rather than explicitly formatting triplets of (query, positive, negative), MNRL requires only pairs.

If a batch contains four query-document pairs, the model computes a 4×4 similarity matrix.

The diagonal elements ($Q_1 \rightarrow D_1$, $Q_2 \rightarrow D_2$, etc.) represent the positive pairs. All off-diagonal elements (e.g., Q_1 compared against D_2, D_3 , and D_4) are dynamically treated as negative pairs⁶.

This implementation relies on the statistical assumption that D_2 is fundamentally irrelevant to Q_1 . In highly specialized domains, this occasionally results in a "false negative" collision, where a randomly grouped document actually happens to answer another query in the batch.

However, the computational efficiency of acquiring $N - 1$ negatives "for free" in every batch of size N overwhelmingly compensates for the noise introduced by occasional false negatives².

16. Matryoshka Representation Learning

Matryoshka Representation Learning is the core algorithmic mechanism enabling the cost-efficiency thesis of this project. Standard contrastive training calculates loss solely on the final, full 768-dimensional vector output. MRL subverts this by instructing the model to calculate the loss multiple times across different truncations of that same vector before performing backpropagation¹.

During the training flow, the model generates a standard 768-dimensional embedding. The MatryoshkaLoss wrapper then extracts the first 768 dimensions and computes the MNRL. It subsequently extracts the first 512 dimensions, computes the MNRL again, and repeats this for 256 and 128 dimensions. The final combined loss is a weighted sum of these discrete losses, which is then used to update the model. By enforcing optimization at every truncation step, the model is architecturally forced to front-load its most critical semantic variance into the earliest dimensions (the first 128 numbers), behaving like nesting dolls.

At inference time, the mechanics are elegantly simple. The transformer still outputs a 768-dimensional vector, meaning MRL does not reduce the embedding generation compute¹. However, the system simply applies array slicing to the output (e.g., `vector[:128]`) and stores only the first 128 dimensions in the vector database. This reduces storage by 83% and speeds up cosine similarity searches proportionately, without requiring the deployment of a separate, smaller model architecture.

17. Experimental Baselines

To prove the efficacy of the pipeline definitively, six distinct baselines must be evaluated against the SciFact test set under identical conditions.

- **Baseline A:** Pretrained model — full 768 dimensions. This establishes the out-of-the-box capability.
- **Baseline B:** Fine-tuned model (Standard MNRL, no MRL) — full 768 dimensions. This isolates and proves that domain fine-tuning alone is effective.
- **Baseline C:** Matryoshka fine-tuned model — full 768 dimensions. This proves that adding the MRL objective did not catastrophically degrade the full-scale representation².
- **Baseline D:** Matryoshka fine-tuned model — 512 dimensions.
- **Baseline E:** Matryoshka fine-tuned model — 256 dimensions.
- **Baseline F:** Matryoshka fine-tuned model — 128 dimensions. This serves as the ultimate test of the quality-to-cost trade-off.

18. Experiment Design

A rigorous experimental matrix must be constructed to log the performance of every baseline and dimension.

Model Strategy	Dimension	Recall@10	MRR@10	nDCG@10	Storage per 1M docs (MB)	Avg Retrieval Latency (ms)

Pretrained BGE	768				3,072 MB	
Standard FT	768				3,072 MB	
Matryoshka FT	768				3,072 MB	
Matryoshka FT	512				2,048 MB	
Matryoshka FT	256				1,024 MB	
Matryoshka FT	128				512 MB	

The retrieval metrics (Recall, MRR, nDCG) will be dynamically collected using the InformationRetrievalEvaluator. The storage size will be calculated theoretically using the formula: Dimension * 4 bytes (for Float32) * Number of Documents. The retrieval latency will be measured empirically by utilizing Python's time.perf_counter() over 100 random queries executed against the FAISS index, isolating the search time from the embedding generation time.

19. Cost Analysis

The cost analysis contextualizes the abstract mathematical results into actionable business intelligence. Embedding generation cost remains a fixed ceiling, as the Transformer always processes the full textual context regardless of the final sliced vector size.

However, vector storage cost decreases strictly linearly with dimensionality. Assuming standard 32-bit floating-point (FP32) precision, a 1-million document corpus at 768 dimensions requires roughly 3 GB of memory. Reducing this to 256 dimensions cuts the requirement to 1 GB, and 128 dimensions reduces it to a mere 512 MB. When scaled to a 10-million document enterprise deployment, moving from 768d (30 GB) to 128d (5 GB) fundamentally shifts the hardware requirement from an expensive multi-node distributed database architecture to a process capable of running entirely in the RAM of a single standard application server.

Vector search computational cost is also reduced linearly during exact exhaustive search (FlatL2), as fewer mathematical operations are required per vector comparison. Furthermore, the reduced memory footprint significantly alleviates memory bandwidth bottlenecks, which are often the primary constraining factor for high-throughput CPU-based inference serving

systems⁷. If an organization achieves an 83% reduction in infrastructure costs while sacrificing only 2-3% of Recall@10 accuracy, the MRL implementation represents a massive, undeniable Return on Investment (ROI).

20. Retrieval Evaluation

The system's search capability is evaluated using industry-standard statistical metrics, which provide different lenses on performance.

Recall@K measures the raw proportion of actual relevant documents present anywhere within the top-K retrieved results. If a query possesses one correct document in the dataset, and it appears at rank 8 in the search results, Recall@10 is 1.0 (100%). MRR (Mean Reciprocal Rank) focuses strictly on the topmost correct answer; it calculates the average of the reciprocal of the rank of the first relevant document. If the document is rank 1, the score is 1.0; if rank 5, the score is 0.2. nDCG (Normalized Discounted Cumulative Gain) evaluates the overall ranking quality of the entire list, applying varying algorithmic weights for different levels of relevance and discounting them logarithmically the further down the list they appear.

For first-stage RAG systems, Recall@K is overwhelmingly the most critical primary metric⁹.

Unlike traditional search engines where users rarely click past the first link (making MRR vital), LLMs programmatically process the entire context window containing all top-K documents. As long as the correct document is present in the context (high Recall), modern LLMs with robust attention mechanisms can extract the necessary information, regardless of whether it was positioned at rank 1 or rank 9.

21. RAG System

To demonstrate end-to-end functionality beyond mere evaluation metrics, a minimal Viable Product (MVP) RAG pipeline will be instantiated.

The architecture flows sequentially: A User Query is encoded by the Matryoshka Embedding Model, and the output vector is immediately truncated to 256 dimensions. This low-dimensional vector is passed to a FAISS index to perform a high-speed similarity search, returning the top 5 candidate documents. These documents are concatenated into a formatted Context String. The context and the original query are bundled into a prompt template and submitted to a generative LLM—such as a lightweight Hugging Face model or a fast API call to simulate edge deployment constraints—which generates the final natural language answer. While production architectures heavily favor employing a Cross-Encoder reranker to re-sort the high-recall candidates returned by the bi-encoder⁸, this step will be omitted for the MVP. Cross-encoders require massive computational overhead, and omitting them maintains strict experimental focus on the bi-encoder dimensionality thesis.

22. Project Architecture

The repository will be structured to rigorously separate engineering concerns, allowing the analytical notebooks to remain uncluttered by deep function definitions or class structures.

matryoshka-rag-project/ | — data/ # Persistent storage for datasets and qrels | — configs/ #
YAML files for model hyperparameters | — src/ | — data_loader.py # BEIR parsing and
PyTorch DataLoaders | — model.py # Model loading and dynamic truncation wrappers |

|— trainer.py # The encapsulated PyTorch training loop | |— evaluate.py # Recall/MRR/nDCG calculation scripts | |— rag_pipeline.py # End-to-end FAISS + LLM generation logic |— notebooks/ # Numbered Colab execution notebooks |— requirements.txt # Exact pip dependencies for reproducibility |— README.md # Final compiled project documentation

This structure ensures that code executed in Google Colab notebooks directly imports reusable Python modules from the src/ directory, reflecting professional software engineering standards.

23. Colab Notebook Structure

The execution pipeline will be segmented across specialized notebooks to manage Colab's ephemeral runtimes and memory limits effectively.

- 01_setup_and_data.ipynb: Orchestrates environment setup, mounts Google Drive, installs dependencies, downloads SciFact, and generates the necessary train/test splits.
- 02_baseline_evaluation.ipynb: Loads the pretrained BGE-base model, indexes the document corpus, and evaluates baseline metrics using the standard evaluator.
- 03_matryoshka_finetuning.ipynb: Configures the data loaders, initializes MNRL and MatryoshkaLoss, executes the primary training loop, and systematically saves checkpoints directly to Google Drive.
- 04_evaluation_and_experiments.ipynb: The core analytical engine. It loads the fine-tuned model, iterates programmatically over dimension sizes (768 down to 64), collects metric data, measures retrieval latency, and structures the output into a pandas dataframe.
- 05_cost_analysis_and_rag.ipynb: Ingests the dataframe to generate matplotlib visualizations comparing dimensions against recall and storage, and executes the final LLM conversational test.

24. Required Libraries

The following requirements.txt identifies the minimum libraries necessary, avoiding bloat to ensure swift installation on Colab instances.

```
torch>=2.0.0 sentence-transformers>=2.5.0 transformers>=4.38.0 datasets>=2.18.0  
faiss-cpu>=1.7.4 numpy pandas matplotlib scikit-learn
```

25. Exact 7-Day Plan

This execution schedule is optimized for part-time, realistic execution (3-4 hours daily) operating within the constraints of Google Colab.

Day 1: Environment & Baseline Data

- **Learning:** Dense retrieval mechanics, Cosine similarity math.
- **Implementation Tasks:** Setup the Colab environment, mount Google Drive, utilize Hugging Face to download the BEIR SciFact dataset.
- **MUST Finish:** A fully accessible dataset residing on persistent Drive storage.

Day 2: Baseline Evaluation

- **Learning:** Recall@K, MRR, FAISS indexing strategies.
- **Implementation Tasks:** Construct the FAISS index from the pretrained embeddings, execute test queries, and run the InformationRetrievalEvaluator.

- **MUST Finish:** Baseline metrics strictly recorded in a dataframe to benchmark future progress.

Day 3: Training Preparation

- **Learning:** Contrastive learning dynamics, MNRL, positive pair formats.
- **Implementation Tasks:** Format the SciFact data into InputExample pairs and establish the PyTorch DataLoader.
- **Colab Tasks:** Begin Notebook 03. Execute a dry run to ensure data loads without memory errors.

Day 4: Matryoshka Fine-Tuning

- **Learning:** Matryoshka Representation Learning theoretical underpinnings¹.
- **Implementation Tasks:** Define MatryoshkaLoss, configure AdamW and CosineAnnealingLR, and execute the training loop for 1-3 epochs.
- **Backup Plan:** If Colab disconnects, the code must be capable of resuming from the latest epoch checkpoint saved to Drive.

Day 5: Dimensionality Experiments

- **Learning:** Dimensionality reduction tradeoffs and indexing structures.
- **Implementation Tasks:** Load the fine-tuned MRL model. Programmatically extract and evaluate vectors at 768, 512, 256, and 128 dimensions.
- **MUST Finish:** The complete experimental matrix populated with empirical data.

Day 6: Visualization and RAG MVP

- **Learning:** Data visualization, LLM generator integration.
- **Implementation Tasks:** Construct matplotlib charts mapping Recall against Storage. Connect the truncated FAISS output to a simple LLM prompt.
- **Colab Tasks:** Complete Notebook 05.

Day 7: Documentation & Polish

- **Implementation Tasks:** Draft the comprehensive README, clean up local code files, structure the final PDF report, and format the project for GitHub deployment.
- **MUST Finish:** Final PDF exported and repository pushed.

26. Implementation Order

1. **Create Colab Environment:** Mount Google Drive, verify GPU allocation via !nvidia-smi.
2. **Download Dataset:** Pull BEIR SciFact via the Hugging Face datasets library.
3. **Prepare Baseline Embeddings:** Encode the SciFact corpus using the native BAAI/bge-base-en-v1.5 weights.
4. **Build FAISS Retrieval:** Index the 768d baseline vectors using FlatL2.
5. **Evaluate Baseline:** Measure Recall@10 and MRR@10. Validate the evaluation script for correctness.
6. **Prepare Query-Document Pairs:** Map the SciFact training queries to their known positive corpus documents.
7. **Initialize Loss Functions:** Instantiate MultipleNegativesRankingLoss and strictly wrap it within MatryoshkaLoss¹.
8. **Fine-Tune Embedding Model:** Execute the Sentence-Transformers fit() function utilizing Mixed Precision.

9. **Save Checkpoints:** Ensure model weights write to Google Drive safely after each epoch.
10. **Truncate Embeddings:** Implement a helper function to dynamically slice generated vectors (`emb[:dim]`).
11. **Evaluate Dimensions:** Run the standard evaluation loop for each truncation step (768, 512, 256, 128).
12. **Measure Storage and Latency:** Log vector array byte sizes and FAISS query execution times.
13. **Build Simple RAG:** Pipe the top-K retrieved text chunks to a basic text generation script.
14. **Create Graphs:** Plot metric curves against dimensional sizes.
15. **Document Results:** Populate the final markdown files with empirical analysis.

27. Compute and Time Management

Given the ephemeral nature of Colab runtimes, robust time and state management is essential. All model artifacts and processed datasets must be written to `/content/drive/MyDrive/...`. If the runtime disconnects, training artifacts remain secure on disk. During Days 3-4, the training loop must be validated using a highly truncated development subset (e.g., 50 examples). The engineer must verify that the loss reliably decreases on this subset before committing to a full 1-hour training run over the entire dataset.

To avoid wasting GPU allocations, developers should close idle Colab tabs. Execution of the FAISS evaluation scripts can often be performed on CPU instances if GPU limits are hit, as CPU-based FAISS inference is sufficiently performant for a 5,000-document dataset.

If CUDA Out of Memory occurs during training, the batch size must be reduced immediately (e.g., from 64 to 32 or 16), and `torch.cuda.empty_cache()` must be called¹². To maintain the algorithmic benefits of large batch sizes on MNRL without exceeding VRAM, gradient accumulation must be implemented—performing multiple forward passes and accumulating the gradients in memory before executing a single optimizer step¹².

28. Failure Modes

Symptom	Cause	Solution	Fallback
CUDA OOM	Batch size is too large for the allocated GPU VRAM.	Reduce batch size; enable Mixed Precision (FP16/BF16); use gradient accumulation.	Switch to the smaller MiniLM base model.
Colab Disconnect	Inactivity timeout or preemptive resource reclamation by	Ensure checkpointing code writes to Drive every 10% of	Resume execution from the latest Drive checkpoint.

	Google.	training steps.	
Poor Baseline Scores	Bug in indexing or evaluation code (e.g., wrong distance metric).	Verify cosine similarity logic aligns with FAISS L2 metric math.	Fall back to using the native InformationRetrievalEvaluator class directly.
Model Collapse	Learning rate is set too high, destroying pre-trained weights.	Lower LR to $1 \times$, add extended warmup steps for the optimizer ¹³ .	Revert to default AdamW parameters.
No MRL Benefit	MatryoshkaLoss dimensional targets or weights misconfigured.	Ensure loss dimensional target arrays are correctly passed to the wrapper ¹ .	Retrain utilizing equal weights (1.0) for every dimension step.
False Negatives	Domain concepts in a batch are heavily overlapping, confusing MNRL.	Increase total dataset size or implement hard negative mining.	Accept the minor metric penalty; MNRL's statistical robustness usually handles it natively ⁶ .

29. Expected Results

The practitioner must analyze the final experimental matrix for specific empirical patterns rather than inventing results.

The most critical indicator of success is the **Fine-tuning Improvement**: The 768d Matryoshka fine-tuned model should outperform the 768d Pretrained baseline on the specific domain (SciFact). This validates that the contrastive learning sequence successfully aligned the vector space to the scientific vocabulary.

Regarding **MRL Preservation**, the 512d and 256d Matryoshka variants should retain $> 95\%$ of the 768d model's Recall@10 score. This proves the core thesis that multi-granularity training forces vital information into early dimensions. A **Degradation Point**—a sharp, non-linear drop-off in retrieval quality—is expected to occur around 64 dimensions, as the dimensional bottleneck becomes too narrow to encode complex scientific semantics.

The **Storage Trade-off** provides the ultimate validation. At 128 dimensions, vector storage costs demonstrably drop by 83% . If the Recall drops by only $3 - 4\%$ at this dimension, the

experiment is a definitive success. An "Excellent result" demonstrates a 256d fine-tuned vector unequivocally beating a 768d un-tuned vector, proving that high-quality domain adaptation allows for massive dimensionality reduction, achieving both higher accuracy and lower cost simultaneously.

30. Visualizations

The final documentation must feature specific data visualizations to communicate the findings clearly.

1. **Recall@10 vs Embedding Dimension:** A line chart exhibiting a gentle slope from 768 down to 128, followed by a steep cliff at 64 dimensions, illustrating where MRL fails.
2. **MRR vs Embedding Dimension:** A corresponding line chart tracking rank position degradation across dimensions.
3. **Storage vs Dimension:** A bar chart demonstrating the linear, predictable decrease in Megabytes required for index storage.
4. **Retrieval Latency vs Dimension:** A line chart showing faster query times (in milliseconds) at lower dimensions, highlighting computational savings.
5. **Baseline vs Fine-Tuned vs Matryoshka:** A grouped bar chart comparing the three 768d models, explicitly proving that domain adaptation improves raw accuracy over the baseline.
6. **Quality vs Storage Tradeoff:** A scatter plot where the X-axis maps Storage Size and the Y-axis maps Recall. This graph allows practitioners to visually identify the Pareto optimal point (likely at 256d or 128d).

31. Final README

The GitHub repository must feature a professional README.md structured specifically for portfolio review:

- **Project Title:** Cost-Efficient Domain RAG with Fine-Tuned Matryoshka Embeddings.
- **One-line Description:** Reducing RAG vector storage by 83% while maintaining domain retrieval accuracy using Matryoshka Representation Learning.
- **Problem / Motivation:** High-dimensional embeddings cause severe infrastructure bloat and latency in scale-out enterprise RAG systems.
- **Key Idea:** Train a nested, multi-dimensional embedding space.
- **Dataset:** SciFact (BEIR).
- **Training Methodology:** MultipleNegativesRankingLoss paired with MatryoshkaLoss fine-tuning the BGE-base architecture.
- **Experimental Setup:** Baselines, Metrics (Recall@10), Truncation steps.
- **Results:** Key metrics matrix highlighting the 256d sweet spot.
- **Cost Analysis:** Financial and infrastructural implications for production deployment.
- **Google Colab Instructions:** Exact commands to reproduce the experiment locally.
- **Limitations & Future Work:** Discussion on the integration of Cross-Encoder rerankers to rescue MRR drops⁹.

32. Interview Preparation

To ensure deep theoretical comprehension, the engineer must be capable of fluently

answering the following questions during technical interviews.

Easy Difficulty:

- **What is an embedding?** A dense array of continuous numbers representing the semantic meaning of text, allowing machines to compute contextual similarity.
- **What does vector dimension mean?** The length of the array; the exact number of individual floating-point values utilized to represent the concept.
- **What is cosine similarity?** A metric calculating the cosine of the angle between two vectors to determine their similarity, mathematically normalized by their magnitude.
- **What is dense retrieval?** Searching a database by computing distances between dense embeddings rather than relying on exact keyword overlaps (sparse retrieval).
- **What is a RAG pipeline?** Retrieval-Augmented Generation retrieves factual context from a database to append to an LLM prompt, grounding its response and mitigating hallucinations.
- **What is fine-tuning?** Adjusting the pre-trained weights of a generalized neural network on specialized data to improve performance on a specific downstream task.
- **What is a positive pair in contrastive learning?** A query and a document that correctly answers or relates to that query, serving as the ground truth.
- **What is FAISS?** Facebook AI Similarity Search, an industry-standard library optimized for highly efficient vector similarity clustering and searching at scale.
- **What does Recall@10 mean?** The statistical percentage of times the correct relevant document appeared anywhere within the top 10 search results.
- **Why reduce embedding dimensions?** To significantly save RAM, persistent disk space, and computational CPU cycles during similarity searches.

Medium Difficulty:

- **Explain MultipleNegativesRankingLoss.** It is a highly efficient contrastive loss function that evaluates a batch of positive pairs, treating the positive documents of all other queries in that batch as negative examples for the current query.
- **What are in-batch negatives?** Utilizing existing elements within the current training batch as negative examples, entirely circumventing the computational and IO costs of processing separate, explicit negative documents⁶.
- **What is the "false negative" problem in MNRL?** When a batch randomly contains two queries that share semantic overlap, the positive document for Query B might actually be highly relevant to Query A. The loss function incorrectly punishes the model for this similarity.
- **How does Matryoshka Representation Learning work?** It calculates the loss across multiple predefined nested subsets of the full vector (e.g., first 128, 256, 512 dimensions) simultaneously during training, mathematically forcing the model to front-load semantic density¹.
- **Does MRL reduce embedding generation time?** No. The transformer architecture still executes its full forward pass. Truncation happens solely as an array slice *after* the full vector is output¹.
- **Explain MRR vs nDCG.** MRR calculates a score based strictly on the rank of the *first*

relevant document found. nDCG evaluates the entirety of the retrieved list, applying varying scores for different levels of relevance and discounting them logarithmically by their rank position.

- **Why use AdamW instead of Adam?** AdamW decouples weight decay (L2 regularization) from the gradient update step, which yields superior regularization and training stability for complex Transformer models¹³.
- **What is the impact of batch size on MNRL?** Larger batch sizes exponentially increase the absolute number of in-batch negatives, creating a substantially harder and more effective contrastive learning environment².
- **Why is Recall more important than MRR in initial RAG retrieval?** Because the LLM receives all top-K documents concurrently in its context window. As long as the correct document is present (Recall), modern LLM attention mechanisms can extract the answer, regardless of whether the document was rank 1 or rank 5⁸.
- **How do you slice a vector in PyTorch or NumPy?** By utilizing standard array slicing syntax, such as `vector[:128]`.

Hard Difficulty:

- **How is the Matryoshka Loss combined during backpropagation?** It is implemented as a weighted sum of the standard contrastive loss calculated at each specified dimensionality step. PyTorch accumulates the gradients across all subset losses prior to stepping the optimizer².
- **What is the architectural difference between a Bi-encoder and a Cross-encoder?** Bi-encoders process the query and document entirely separately into independent vectors, allowing for pre-computation and fast retrieval. Cross-encoders concatenate them and process them together through the transformer layers, allowing cross-attention. This yields maximum accuracy but at massive computational cost, rendering them suitable only for reranking small candidate sets, never for initial retrieval⁸.
- **If you truncate a standard (non-MRL) embedding, what occurs?** The semantic representation instantly collapses. Standard models distribute semantic variance relatively isotropically across all dimensions. Truncating arbitrarily discards critical information, effectively destroying retrieval accuracy.
- **How does gradient accumulation resolve limited GPU VRAM?** It allows a system to simulate a massive batch size by performing the forward and backward passes on small micro-batches that fit in VRAM, accumulating the gradients computationally, and only stepping the optimizer after N micro-batches have been processed¹².
- **What is catastrophic forgetting during fine-tuning?** A phenomenon where a neural network learns the new domain distribution so aggressively that its weights overwrite the generalized knowledge gained during initial pre-training, destroying its performance on out-of-domain tasks.
- **Why use Mixed Precision (FP16/BF16) training?** It utilizes 16-bit floating-point types instead of 32-bit types for specific matrix operations, cutting memory consumption in half and leveraging specialized GPU tensor cores to dramatically accelerate training, with

negligible impact on gradient accuracy¹².

- **What is temperature scaling in contrastive loss?** A hyperparameter that scales the raw logits prior to the softmax operation, manipulating the sharpness of the probability distribution, which assists the model in separating hard negatives from positive pairs during training.
- **How would you scale this architecture to a billion documents?** By deploying a highly optimized Approximate Nearest Neighbor (ANN) index such as HNSW or IVF-PQ in FAISS, heavily relying on MRL-reduced vectors (e.g., 64d or 128d) to guarantee the massive index physically fits into the clustered RAM of the distributed database.
- **What is the specific effect of the learning rate warmup phase?** It deliberately and slowly increases the learning rate from 0 to the predefined maximum value over the initial iterations, preventing large, chaotic gradient updates from irreversibly shattering the delicately pre-trained weights early in the fine-tuning process¹³.
- **Can MRL be combined with dynamic quantization architectures?** Yes. A 768d FP32 vector reduced to 128d via MRL can subsequently be dynamically quantized to INT8 data types. This stacks both algorithmic representation compression and hardware-level precision compression to achieve unprecedented infrastructure savings, particularly when served via optimized CPU backends like ONNX Runtime⁷.

33. Research Sources Analysis

The architectural methodology of this project is strictly validated by current AI engineering literature and Natural Language Processing research.

The core application of MRL relies heavily on the original principles laid out by Kusupati et al. regarding Matryoshka Representation Learning, which dictates forcing models to learn multi-granularity representations. Recent applications in models like the multilingual encoder mmBERT-small confirm that utilizing Matryoshka learning paired with massive contrastive batch sizes yields highly efficient truncated vectors. These models frequently lose less than 2% overall retrieval quality when truncating from high dimensions down to 256 dimensions².

Furthermore, the mechanics of MultipleNegativesRankingLoss serve as the industry standard for fine-tuning bi-encoders. Sentence-Transformers documentation and community implementations clarify that manually flattening dataset structures allows for strict, programmatic control over batch grouping, which is absolutely critical when attempting to properly leverage in-batch negatives⁶. As Neelakantan et al. noted, large-batch contrastive learning boosts retrieval quality by exponentially increasing the probability of the network encountering mathematically difficult hard negatives².

For model selection, the BGE architecture family (BAAI/bge-base-en-v1.5) remains highly competitive. However, benchmarks like MTEB must be interpreted cautiously by practitioners; evaluation on domain-specific datasets (like BEIR's SciFact) provides a much stronger, less contaminated signal of actual enterprise RAG performance than generic web-corpus averages¹.

Regarding infrastructural hardware optimization, dynamic quantization to INT8 and serving via ONNX Runtime provides significant throughput multipliers on standard CPUs compared to

native PyTorch implementations⁷. While dynamic quantization is technically a post-training deployment strategy, combining it with the algorithmic dimension truncation of MRL represents the absolute cutting edge of cost-efficient AI deployment. Finally, standard production training practices—such as utilizing AdamW, CosineAnnealingLR, and mixed precision casting—ensure stable mathematical optimization and rapid convergence during the fine-tuning loops¹².

34. PDF Document Requirements

This documentation has been architected strictly to meet professional PDF export standards. It utilizes hierarchical Markdown formatting, numerical sectioning, and structured data tables to ensure clarity. The logical flow advances seamlessly from high-level executive summaries to rigorous mathematical formalizations and exact implementation steps, fulfilling the structural requirement while maintaining the continuous, fluid narrative expected of an advanced AI engineering blueprint.

35. Final "Definition of Done"

The project is considered fully complete and ready for portfolio presentation when the following checklist is executed:

- ☐ Google Colab environment configured with Google Drive persistent mounting.
- ☐ BEIR SciFact dataset downloaded, inspected, and split correctly.
- ☐ Pretrained baseline model evaluated against the test queries.
- ☐ Baseline Recall@10, MRR, and nDCG metrics definitively recorded.
- ☐ PyTorch data loaders configured for positive-pair ingestion.
- ☐ MultipleNegativesRankingLoss correctly instantiated in code.
- ☐ MatryoshkaLoss implemented with accurate target dimensionalities.
- ☐ Embedding model successfully fine-tuned (1-3 epochs) without model collapse.
- ☐ Fine-tuned model evaluated at 768 dimensions.
- ☐ Fine-tuned model evaluated at 512 dimensions.
- ☐ Fine-tuned model evaluated at 256 dimensions.
- ☐ Fine-tuned model evaluated at 128 dimensions.
- ☐ Vector storage sizes theoretically calculated and logged.
- ☐ Empirical retrieval latency measured and logged for all dimensions.
- ☐ Data successfully visualized via Matplotlib (Recall vs. Dimension, Storage vs. Dimension).
- ☐ Simple generative LLM pipeline established using the truncated FAISS output.
- ☐ GitHub repository structured according to strict architectural specs.
- ☐ README.md authored with final experiment metrics and visualizations.
- ☐ Interview questions reviewed and comprehensively understood.
- ☐ Final documentation exported as a professional PDF.

Works cited

1. How to Build a Matryoshka-Optimized Sentence Embedding Model,
<https://www.marktechpost.com/2026/02/11/how-to-build-a-matryoshka-optimiz>

[ed-sentence-embedding-model-for-ultra-fast-retrieval-with-64-dimension-truncation/](#)

2. Bekko Embedding: Parameter-Efficient Multilingual Retrieval ... - arXiv, <https://arxiv.org/html/2607.25180>
3. AI Engineer Salary in India 2026 - EICTA Consortium - IIT Kanpur, <https://www.eicta.iitk.ac.in/knowledge-hub/artificial-intelligence/ai-engineer-salary-india-2026>
4. Highest Paying AI Careers in India: Which Role Fits You Best?, <https://www.microdegree.work/blog/highest-paying-ai-careers-in-india>
5. AI Engineer Salary in India 2026: Fresher to Experienced Guide, <https://www.craw.in/ai-engineer-salary-in-india>
6. Custom batches in sentence-transformers for, <https://discuss.huggingface.co/t/custom-batches-in-sentence-transformers-for-multiplenegativesrankingloss/175587>
7. Efficient INT8 Inference of Small NLP Models on Server CPUs with, <https://arxiv.org/pdf/2608.18182>
8. Best Embedding Model for RAG: What You Need to Know, <https://unstructured.io/blog/understanding-embedding-models-make-an-informed-choice-for-your-rag>
9. Bi-Encoders vs Cross-Encoders — ZeroEntropy Blog, <https://zeroentropy.dev/articles/biencoder-vs-crossencoder/>
10. Sentence Transformers in Python: Train Retrieval Embeddings with, <https://www.youtube.com/watch?v=l3wFPbxeFkM>
11. Embedding Models in 2026: Choosing, Benchmarking ... - Tensoria, <https://tensoria.fr/en/blog/embedding-models-2026-guide>
12. PyTorch Training Loop: A Production-Ready Template for AI, <https://genmind.ch/posts/PyTorch-Training-Loop-A-Production-Ready-Template-for-AI-Engineers/>
13. How to Speed Up Training Convergence of Language Models, <https://machinelearningmastery.com/how-to-speed-up-training-of-language-models/>
14. [D] How would you diagnose these spikes in the training loss? - Reddit, https://www.reddit.com/r/MachineLearning/comments/1cf4gw9/d_how_would_you_diagnose_these_spikes_in_the/
15. ONNX vs PyTorch Speed: In-Depth Performance Comparison - Dev-kit, <https://dev-kit.io/blog/machine-learning/onnx-vs-pytorch-speed-comparison>
16. Accelerating Deep Learning Inference: A Comparative Analysis of, <https://www.mdpi.com/2079-9292/14/15/2977>
17. Optimizing Transformer Inference with ONNX Runtime and ... - Medium, <https://medium.com/@bhagyarana80/optimizing-transformer-inference-with-onnx-runtime-and-quantization-098f8149a15c>
18. Scaling-up PyTorch inference: Serving billions of daily NLP, <https://opensource.microsoft.com/blog/2022/04/19/scaling-up-pytorch-inference-serving-billions-of-daily-nlp-inferences-with-onnx-runtime/>
19. junaidaliop/zij: A canon of deep learning optimizers. - GitHub,

<https://github.com/junaidaliop/zij>